



Northern University

Of Business and Technology Khulna

LAB Report 3

Course Code: CSE 1202

Course Title: Object Oriented Programming Lab

Submitted by:

Name : Nazmus Sakib
ID : 11220320888
Section: 2D
Department of Computer Science and
Engineering

Submitted to:

Name: Faria Afrin Tisha
Designation: Lecturer
Department: Computer Science and
Engineering

Date of Submission: 31.03.2026

Experiment No: 03

Experiment Name: Design and Implementation of a Smart Home Control System Using C++

Introduction

A Smart Home Control System is designed to manage and control multiple electronic devices efficiently. In modern programming, Object-Oriented Programming (OOP) plays a vital role in modeling such real-world systems.

This experiment demonstrates the use of core OOP concepts including inheritance, method overriding, method overloading and polymorphism through virtual functions. A base class named Device is created and multiple derived classes such as Light, Fan and AC are implemented to extend its functionality.

Inheritance allows code reuse, overriding enables customized behavior for each device and overloading provides flexibility in method usage. This system simulates how different smart devices can be controlled in a unified structure.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Device {
protected:
    string deviceName;

public:
    Device(string name) : deviceName(name) {}

    virtual ~Device() {}

    void turnOn() {
        cout << deviceName << " is now ON" << endl;
    }

    void turnOff() {
        cout << deviceName << " is now OFF" << endl;
    }

    virtual void status() {
        cout << deviceName << " status unknown" << endl;
    }
};

class Light : public Device {
```

```

private:
    int brightness;

public:
    Light(string name, int b) : Device(name), brightness(b) {}

    void status() override {
        cout << deviceName << " brightness level: " << brightness << endl;
    }
};

class Fan : public Device {
private:
    int speed;

public:
    Fan(string name, int s) : Device(name), speed(s) {}

    void status() override {
        cout << deviceName << " speed level: " << speed << endl;
    }
};

class AC : public Device {
private:
    float temperature;
    string mode;

public:
    AC(string name) : Device(name), temperature(24), mode("Cool") {}

    void status() override {
        cout << deviceName << " temperature: " << temperature
            << ", mode: " << mode << endl;
    }

    void setTemperature(int temp) {
        temperature = temp;
    }

    void setTemperature(int temp, string m) {
        temperature = temp;
        mode = m;
    }
}

```

```
}

void setTemperature(float temp, string m, bool ecoMode) {
    temperature = temp;
    mode = m;
    if (ecoMode)
        cout << "Eco mode enabled" << endl;
}

};

int main() {
    Light light("Living Room Light", 70);
    Fan fan("Ceiling Fan", 3);
    AC ac("Bedroom AC");

    light.turnOn();
    light.status();

    fan.turnOn();
    fan.status();

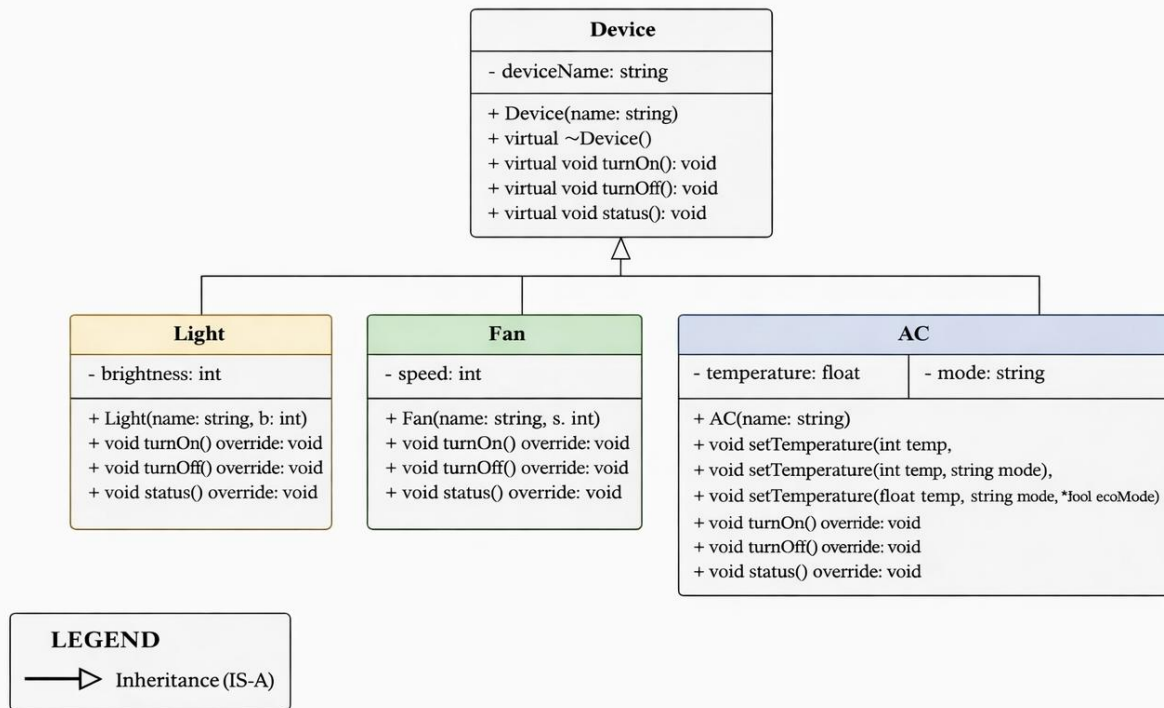
    ac.turnOn();
    ac.setTemperature(22, "Cool");
    ac.status();

    return 0;
}
```

Output:

```
Living Room Light is now ON
Living Room Light brightness level: 70
Ceiling Fan is now ON
Ceiling Fan speed level: 3
Bedroom AC is now ON
Bedroom AC temperature: 22, mode: Cool
```

```
Normal program termination. Exit status: 0
```

UML Diagram:**Discussion and Conclusion**

The experiment was successfully completed by implementing a Smart Home Control System using OOP concepts in C++. The program demonstrates inheritance through base and derived classes, method overriding for customized device behavior and method overloading in the AC class.

The structure of the program is modular, reusable and easy to understand. Each class represents a real-world device, making the system scalable and maintainable. The time complexity is constant for each operation as it performs direct function calls.

In conclusion, this experiment effectively shows how object-oriented principles can be applied to design real-life systems, improving both programming skills and conceptual understanding. The system can be further extended by adding more smart devices such as sensors, security systems, or automation modules, making it more scalable and practical.